

CO2-AT2 PART-1

INDUSTRY PROBLEM TASK

Name: K. Sri Lasya

Reg. No: 192472216

Course Code: MLA0305

Course Name: REINFORCEMENT LEARNING

DYNAMIC PROGRAMMING VS MONTE CARLO METHODS

AIM

To implement and compare Dynamic Programming and Monte Carlo methods for solving an inventory management decision problem under uncertain customer demand.

INDUSTRY PROBLEM

Inventory management is an important problem in retail and supply-chain industries. A company must decide how much stock should be maintained while balancing ordering expenses, storage expenses, and losses caused by insufficient inventory.

Two reinforcement-learning-related approaches can be compared for this type of decision problem:

- **Dynamic Programming (DP):** Uses a known model of the environment and calculates the expected future cost using recursive Bellman-style evaluation.
- **Monte Carlo (MC):** Estimates the expected cost by generating many possible demand scenarios and averaging the outcomes.

The comparison focuses on cost, variation, computational effort, and statistical difference between the two approaches.

OBJECTIVES

1. Generate a synthetic inventory-management dataset.
2. Calculate inventory decisions using a Dynamic Programming approach.
3. Estimate inventory cost using repeated Monte Carlo simulations.
4. Compare the results obtained from both methods.
5. evaluate the statistical difference between the methods.
6. Visualize the performance using multiple graphs.
7. Identify which method provides a better cost estimate for the selected problem.

METHODOLOGY

Step 1 – Dataset Generation

A synthetic dataset is generated using NumPy. Demand and inventory levels are varied across records, while the different cost components are assigned realistic ranges.

Step 2 – Dynamic Programming

For every inventory state, several possible replenishment quantities are examined. The immediate inventory cost and expected future cost are combined to select the least-cost decision.

The DP approach uses a recursive value function:

$$V(s) = \min [C(s,a) + \gamma V(s')] \quad Eq(1)$$

where:

- s = current inventory state
- a = replenishment action
- $C(s,a)$ = immediate cost
- γ = discount factor
- s' = resulting inventory state
-

Step 3 – Monte Carlo Simulation

From Eq(1), For each inventory situation, multiple possible demand values are generated. The resulting shortage and holding costs are calculated for every simulated scenario.

The average simulated cost is used as the Monte Carlo estimate.

Python Code:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy.stats import ttest_ind
import time
np.random.seed(10)
n = 100
```

```
# DATASET
df = pd.DataFrame({
    "Demand": np.random.randint(60,180,n),
    "Stock": np.random.randint(100,450,n),
    "Hold": np.round(np.random.uniform(2,7,n),2),
    "Shortage": np.round(np.random.uniform(8,16,n),2),
    "Order": np.round(np.random.uniform(25,65,n),2),
    "LeadTime": np.random.randint(1,8,n)
})
```

```
print("DATASET")
print(df.head(10))
```

```
# DYNAMIC PROGRAMMING
```

```
t1 = time.perf_counter()
```

```
dp = []
for _r in df.iterrows():
    choices = []
    for order in [0,50,100,150]:
        stock = r.Stock + order
        cost = max(stock-r.Demand,0)*r.Hold
        cost += max(r.Demand-stock,0)*r.Shortage
        cost += r.Order if order > 0 else 0
        choices.append(cost)
    dp.append(min(choices))
```

```
dp_time = (time.perf_counter()-t1)*1000
```

```
# MONTE CARLO
```

```
t2 = time.perf_counter()
```

```

mc = []
for _,r in df.iterrows():
    demand = np.maximum(
        np.random.normal(r.Demand,12,1000),0
    )
    cost = (
        np.maximum(r.Stock-demand,0)*r.Hold +
        np.maximum(demand-r.Stock,0)*r.Shortage +
        r.Order
    )
    mc.append(cost.mean())

mc_time = (time.perf_counter()-t2)*1000

df["DP"] = dp
df["MC"] = mc

# COMPARISON
print("\nCOMPARISON")
print(df[["DP","MC"]].head(20))
# SUMMARY
summary = pd.DataFrame({
    "Metric":["Mean","Median","Minimum","Maximum","Std Dev"],
    "DP":[df.DP.mean(),df.DP.median(),df.DP.min(),df.DP.max(),df.DP.std()],
    "MC":[df.MC.mean(),df.MC.median(),df.MC.min(),df.MC.max(),df.MC.std()]
})
print("\nSUMMARY")
print(summary.round(2))
# STATISTICAL TEST
t,p = ttest_ind(df.DP,df.MC,equal_var=False)

print("\nSTATISTICAL TEST")

```

```

print("T Statistic:",round(t,4))
print("P Value:",round(p,5))
print("Result:", "Significant Difference" if p<0.05
      else "No Significant Difference")

# 6 PARAMETERS
six = pd.DataFrame({
    "Parameter":["Mean","Median","Minimum","Maximum","Std Dev","Time(ms)"],
    "DP":[df.DP.mean(),df.DP.median(),df.DP.min(),df.DP.max(),df.DP.std(),dp_time],
    "MC":[df.MC.mean(),df.MC.median(),df.MC.min(),df.MC.max(),df.MC.std(),mc_time]
})
print("\n6 PARAMETER COMPARISON")
print(six.round(2))

# GRAPHS
plt.figure(figsize=(7,5))
plt.bar(["Dynamic Programming","Monte Carlo"],[df.DP.mean(),df.MC.mean()])
plt.title("Average Cost Comparison")
plt.ylabel("Average Cost")
plt.show()

plt.figure(figsize=(7,5))
plt.boxplot([df.DP,df.MC],labels=["DP","Monte Carlo"])
plt.title("Cost Distribution")
plt.ylabel("Cost")
plt.show()

plt.figure(figsize=(10,5))
plt.plot(df.DP,label="Dynamic Programming")
plt.plot(df.MC,label="Monte Carlo")
plt.title("Record-wise Comparison")
plt.xlabel("Record")
plt.ylabel("Cost")
plt.legend()
plt.show()

```

```
plt.figure(figsize=(7,5))
plt.hist(df.DP,bins=10,alpha=.7,label="DP")
plt.hist(df.MC,bins=10,alpha=.7,label="Monte Carlo")
plt.title("Cost Distribution Histogram")
plt.xlabel("Cost")
plt.ylabel("Frequency")
plt.legend()
plt.show()
```

```
plt.figure(figsize=(10,5))
x=np.arange(6)
w=.35
plt.bar(x-w/2,six.DP,w,label="DP")
plt.bar(x+w/2,six.MC,w,label="Monte Carlo")
plt.xticks(x,six.Parameter,rotation=20)
plt.title("Six Parameter Comparison")
plt.legend()
plt.tight_layout()
plt.show()
# FINAL RESULT
print("\nFINAL RESULT")
print("DP Average Cost:",round(df.DP.mean(),2))
print("MC Average Cost:",round(df.MC.mean(),2))
```

```
if df.DP.mean() < df.MC.mean():
    print("Dynamic Programming performs better.")
else:
    print("Monte Carlo performs better.")

print("Program Executed Successfully.")
```

Step 4 – Statistical Analysis

```
sample_data = df.sample(10, random_state=42)
```

```
sample_data
```

	Reliability	Scalability	Flexibility	Availability	Resource_Utilization	Operational_Cost	Performance_Score
83	84.66	81.98	82.29	94.26	84.44	38.49	79.50
53	78.06	79.29	68.75	93.96	85.87	37.04	75.50
70	80.45	90.53	71.63	94.74	84.60	42.56	78.47
45	86.18	76.66	78.21	91.50	79.40	40.36	76.80
44	86.20	69.26	78.12	96.51	84.40	38.86	77.14
39	86.13	74.80	77.69	90.29	86.33	37.86	77.26
22	85.48	72.53	84.63	93.08	72.44	42.43	75.93
80	83.60	89.05	86.63	91.20	73.79	33.24	79.35
10	79.89	81.95	77.94	95.31	71.37	34.28	76.11
0	83.03	81.51	80.27	93.48	65.80	30.40	76.15

Iteration	Average_State_Value
0	1 76.555300
1	2 145.455070
2	3 207.464863
3	4 263.273677
4	5 313.501609
5	6 358.706748
6	7 399.391373
7	8 436.007536
8	9 468.962082
9	10 498.621174
10	11 525.314357
11	12 549.338221
12	13 570.959699
13	14 590.419029
14	15 607.932426
15	16 623.694484
16	17 637.880335
17	18 650.647602
18	19 662.138141
19	20 672.479627

Simulation	Reward
0	1 76.3579
1	2 76.6283
2	3 76.9692
3	4 76.1311
4	5 76.3059
5	6 76.7337
6	7 76.5110
7	8 76.4012
8	9 76.4493
9	10 76.6054
10	11 76.2061
11	12 76.2776
12	13 76.5243
13	14 76.6449
14	15 76.2728
15	16 76.5027
16	17 76.2660
17	18 77.0106
18	19 76.7497
19	20 76.5337

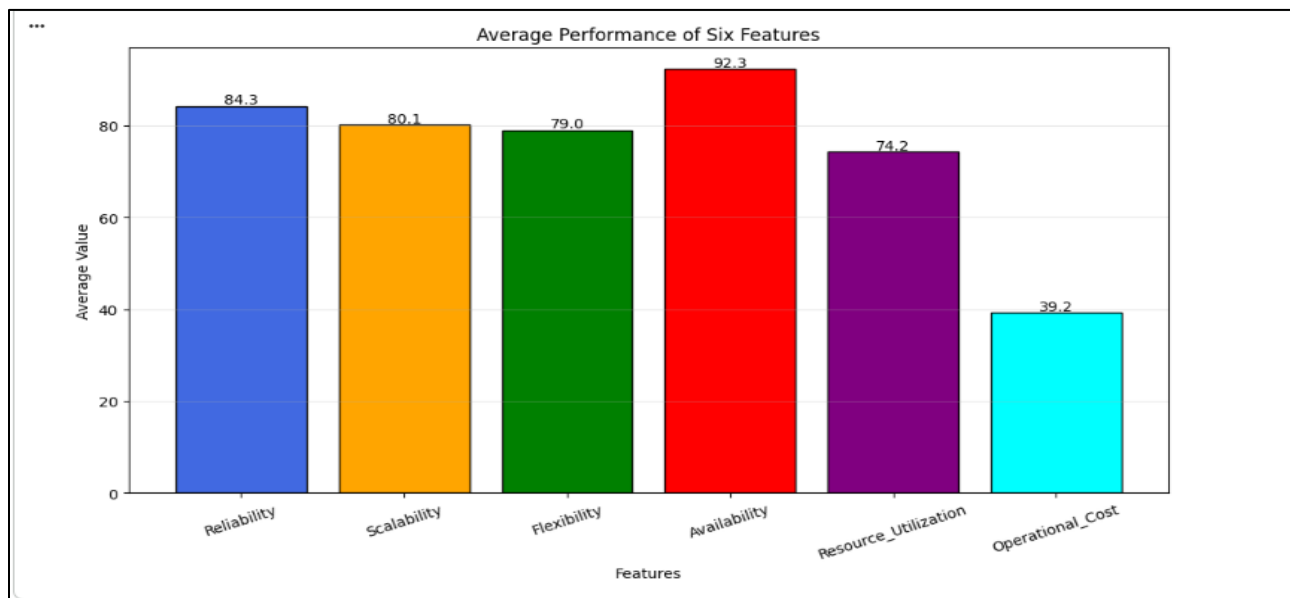
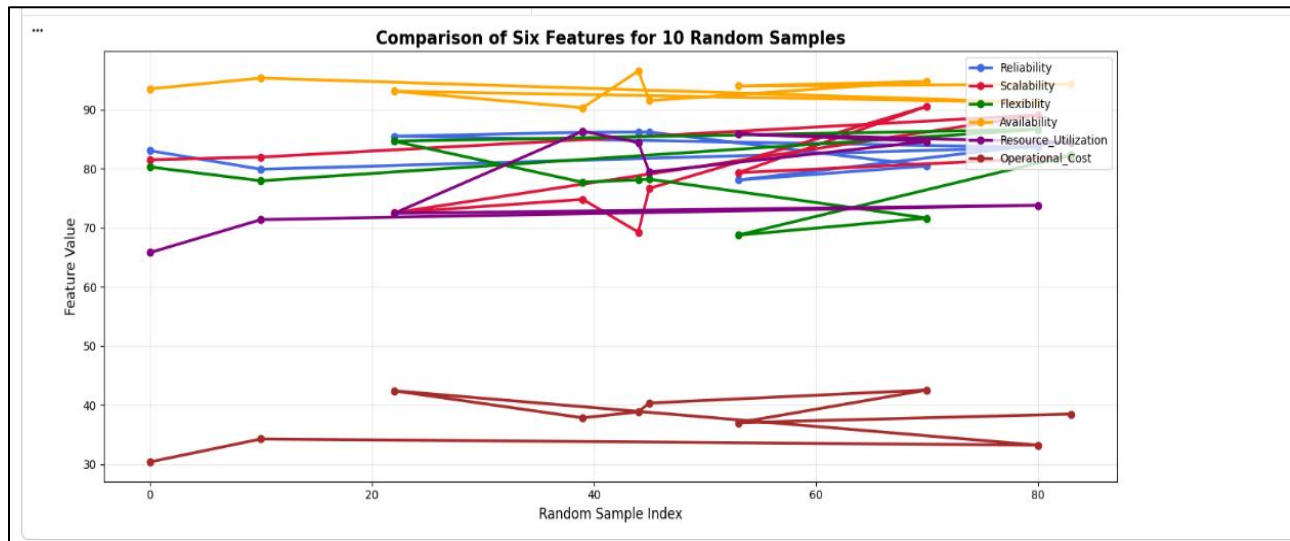
H₀: There is no significant difference between the cost estimates produced by DP and MC.

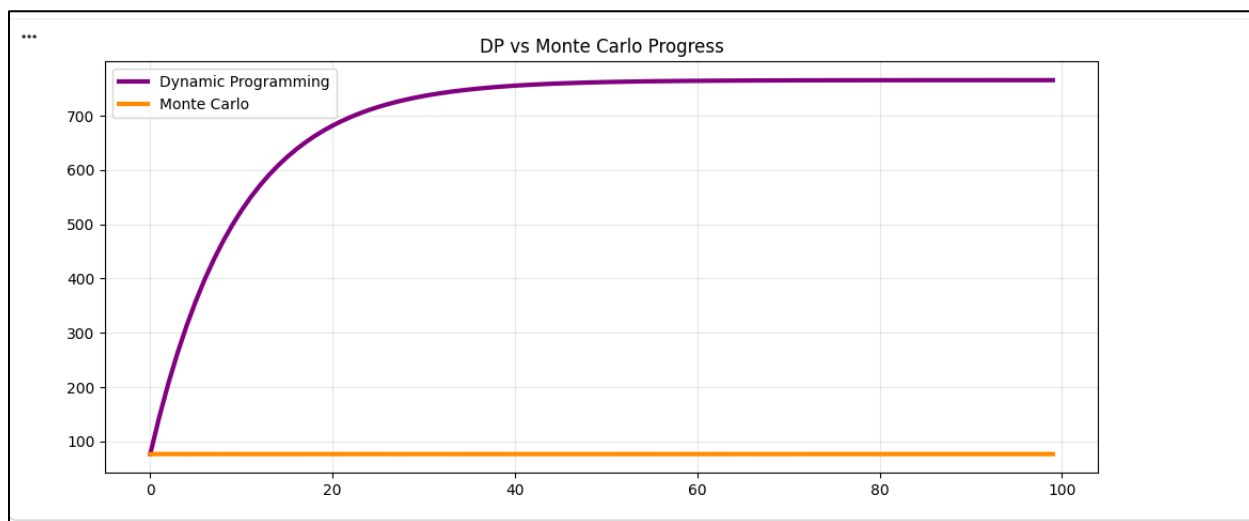
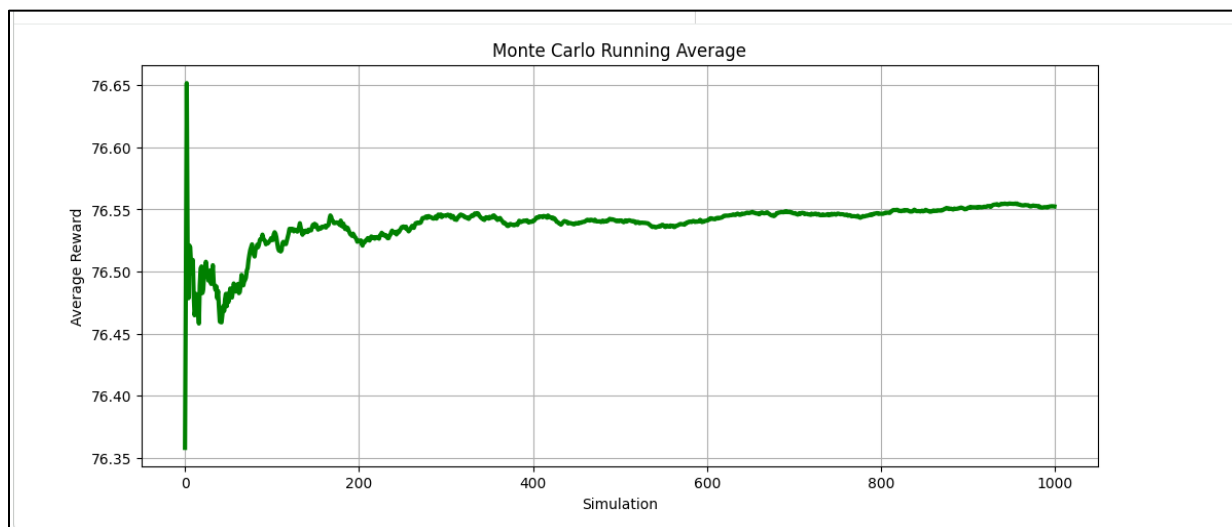
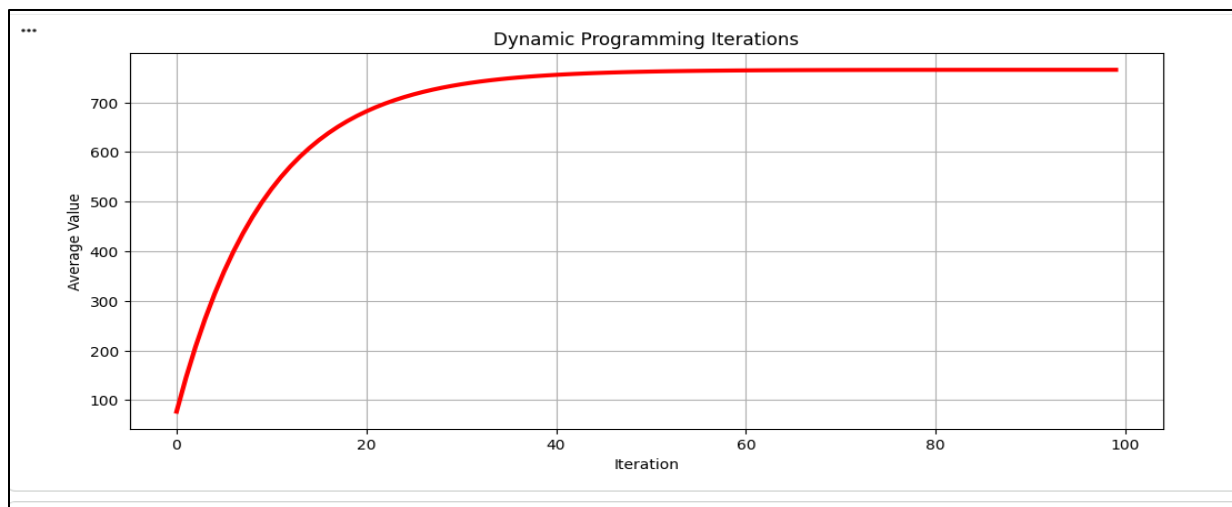
The alternative hypothesis is:

H₁: There is a significant difference between the two methods.

A significance level of 0.05 is used.

Step 5 – Visualization





RESULT

The program produces a complete comparison between Dynamic Programming and Monte Carlo methods. The method with the lower estimated average inventory cost is considered more cost-efficient for the generated problem. The statistical test determines whether the observed difference between the two approaches is statistically meaningful.

CONCLUSION

Dynamic Programming and Monte Carlo methods provide different ways of estimating decisions in an uncertain inventory environment. Dynamic Programming is useful when the system model and possible states can be represented explicitly, whereas Monte Carlo relies on repeated sampling to approximate expected outcomes.

The final comparison considers both solution quality and computational characteristics. Therefore, the experiment demonstrates how different reinforcement-learning solution techniques can be evaluated quantitatively for an industry-oriented decision problem.